September 2025

Mathematical Framework to Enable Adaptive Neuron Transitions

Bentley Yusen LIN [a,1],
[a] *Safeware Technologies Inc., Ltd*
ORCiD ID: Bentley Yusen Lin https://orcid.org/0009-0000-2569-5919

Abstract. This paper introduces a novel mathematical framework for adaptive neural transfer functions that dynamically reconfigure input-output state spaces based on structural gradients. Unlike traditional weight-centric approaches, our Adaptive State-Space Transfer Function (ASSTF) enables neurons to evolve their topology during learning and inference. We formalize continuous gradient-driven dimensionality adaptation, establish theoretical guarantees for state-space continuity and Lyapunov stability, and present a regularized bilevel optimization scheme. This framework provides a mathematical foundation for adaptive neurons that transcend weight-centric learning.

Keywords. Artificial Intelligence, Adaptive Neuron, Dynamic Architecture

1. Introduction

Traditional neural networks suffer from static architectures that limit adaptability to changing data distributions [1]. We propose the Adaptive State-Space Transfer Function (ASSTF), a mathematical framework enabling neurons to dynamically reconfigure their input-output spaces based on structural gradients. This work addresses three fundamental limitations: (1) topological rigidity in neural architectures, (2) discontinuous capacity changes during training, and (3) inference-time inflexibility [2].

2. Related Work

Our work on Adaptive State-Space Transfer Functions (ASSTF) sits at the intersection of several research domains: adaptive neural architectures, dynamic inference, and optimization in deep learning. We contextualize our contribution relative to these fields.

2.1. Neural Architecture Search and Adaptive Networks

Traditional Neural Architecture Search (NAS) [3,4] optimizes over a discrete, combinatorial search space of architectures, a process that is typically computationally prohibitive and requires complete retraining. DARTS [5] and subsequent gradient-based NAS meth-

[1] Corresponding Author: Bentley Yusen Lin, Bentley@safeware.com.tw.

ods differentiably relax this search space but still converge to a single, static final architecture. Our framework is fundamentally different: it forgoes the search for an optimal static architecture in favor of continuous, gradient-driven *adaptation* of each neuron's functional topology throughout both training and inference. This is more closely related to ideas in adaptive activation functions [6] and hypernetwork-based weight generation [7], but with a focus on low-rank structural modifications at the single-neuron level rather than generating weights for entire layers.

2.2. Dynamic Inference and Conditional Computation

Another line of research aims to make inference adaptive or dynamic. This includes methods for early exiting [8], activating different model components based on the input [9], or having dynamic weights [10]. These approaches typically learn a routing policy or a gating function. While ASSTF also employs a gating mechanism (Section 6), it is not for routing but for smoothly phasing in or out the contribution of a neuron's transformed state-space based on its activity. More critically, our method dynamically evolves the *transformation itself* via structural parameters θ_s, a form of capacity adaptation not present in standard dynamic inference models.

2.3. Optimization and Continuous-Time Models

Our theoretical analysis draws from Lyapunov stability theory [11] and bilevel optimization [14], applying them to the novel context of online architectural evolution. The closest philosophical relative is perhaps the line of work on Neural Ordinary Differential Equations (Neural ODEs) [15], which also considers continuous-depth models. However, Neural ODEs fix the width and functional form of the layers, only adapting the depth via numerical integration. In contrast, ASSTF fixes the "depth" (number of layers) but adapts the "width" and functional form of individual neurons over time. Our soft-thresholding projection operator $\mathscr{P}$ [13,18] provides a mathematically grounded method for continuous rank adaptation, a technique common in matrix completion but novel in the context of neuronal function.

2.4. Novelty and Positioning

The novelty of ASSTF lies in its synthesis of these ideas into a unified, neuron-level framework for continuous gradient-driven architectural adaptation. Unlike NAS, it is always-on and requires no separate search phase. Unlike dynamic inference, it adapts the core transformation rather than just a routing path. And unlike Neural ODEs, it adapts structural complexity rather than temporal depth. This provides a new mathematical foundation for building networks that can truly evolve their capacity in response to data.

3. Current Technical Constraints in Neural Weight Adjustment

Traditional neural networks rely exclusively on weight adjustments for learning, governed by the gradient descent rule:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \nabla_{w_{ij}} \mathscr{L} \tag{1}$$

where η is the learning rate and $\mathscr{L}$ the loss function. This approach presents fundamental constraints [16]:

Limitations:

- *Topological rigidity*: Fixed input-output dimensions prevent structural adaptation
- *Discontinuous capacity changes*: Architecture modifications require complete retraining
- *Stationarity assumption*: Fixed structure assumes static data distributions

4. Adaptive State-Space Transfer Function Framework

4.1. Notation

- $\Theta = \{\theta_c, \theta_s\}$: Complete set of all parameters for a neuron
- $\theta_c \in \mathbb{R}^M$: Core parameters (traditional weights and biases)
- $\theta_s \in \mathbb{R}^N$: Structural parameters (control topology of state-space transformation)
- $\Gamma(\mathbf{x}, \theta_c)$: Base transfer function (e.g., linear transformation $\mathbf{x} \cdot \mathbf{W} + b$)
- $\circledast$: Composition operation (output of Γ modulated by output of Ψ)

4.2. ASSTF Definition

Definition 1 (ASSTF)**.** *For neuron i at time t:*

$$\phi^{(i)}(t; \mathbf{x}, \Theta) = \Gamma(\mathbf{x}, \theta_c(t)) \circledast \Psi(\nabla_\Theta \mathcal{L}, \theta_s(t)) \tag{2}$$

where Ψ ensures $\mathscr{C}^1$ continuity in θ_s transitions [13,18,19].

4.3. Continuous State-Space Modulator

Revised to eliminate discrete jumps [13,18]:

$$\Psi(\cdot) = \mathscr{P}\left([\xi(\nabla_{\theta_s}\mathcal{L}) \cdot \mathbf{W}_\tau] \otimes \exp\left(-\zeta \|\nabla_{\theta_s}\mathcal{L}\|_F\right)\right) \tag{3}$$

Components:

- $\mathbf{W}_\tau$: Basis matrix defining space of possible structural transformations
- $\xi(\cdot)$: Structural gradient interpretation function (e.g., tanh)
- ζ: Damping coefficient scaling update based on gradient magnitude
- $\mathscr{P}$: Projection operator enforcing smooth transitions via soft-thresholding

Key improvements:

1. *Soft rank adaptation* via singular value thresholding: The projection operator $\mathscr{P}(\mathbf{M})$ applies a continuous, smooth thresholding function to the singular values of matrix $\mathbf{M}$. This function attenuates singular values below an adaptive threshold $\alpha\|\nabla\mathscr{L}\|$, effectively performing a "soft" dimensionality reduction that is guided by the learning process. This provides a continuous alternative to hard thresholding and is crucial for maintaining $\mathscr{C}^1$ continuity during structural changes [13,21].

$$\mathscr{P}(\mathbf{M}) = \mathbf{U}\widetilde{\Sigma}\mathbf{V}^T \quad \text{where} \quad \widetilde{\sigma}_i = \sigma_i \cdot (1 + \exp(-\beta(\sigma_i - \alpha\|\nabla\mathscr{L}\|)))^{-1} \tag{4}$$

The sigmoidal term acts as a smooth approximation of a soft-thresholding rule, shrinking each singular value σ_i towards zero. This formulation is inspired by and generalizes the principles of singular value thresholding for low-rank matrix estimation [21].

2. *Dynamic projection* with continuity constraint [22]:

$$\min_{\mathbf{A},\mathbf{B}} \left\| \frac{\partial^2 \mathbf{W}_\tau}{\partial t^2} \right\|_F \quad \text{s.t.} \quad \frac{d\mathbf{A}}{dt} = \eta \frac{\partial \mathscr{L}}{\partial \mathbf{W}_\tau} \mathbf{B}^T - \lambda \mathbf{A} \tag{5}$$

4.4. Representational Capacity Analysis

Definition 2 (Adaptive Capacity Metric)**.** *The instantaneous representational capacity of ASSTF layer with n neurons at time t:*

$$\mathscr{C}(t) = \exp\left(n \log \bar{k}(t)\right) \quad \textit{where} \quad \bar{k}(t) = \mathbb{E}_{\mathbf{x}}[\dim(\Psi(t))] \tag{6}$$

Cumulative capacity: $\mathscr{C}_{cum}(n,T) = \int_0^T \mathscr{C}(t)dt$

Theorem 1 (Capacity Scaling)**.** *ASSTF capacity grows superlinearly with operational time T [22]:*

$$\mathscr{C}_{ASSTF}(n,T) = O\left(\exp(\kappa\sqrt{T}) \cdot n^d\right) \quad \textit{for} \quad \kappa > 0, d > 0 \tag{7}$$

while static networks achieve $\mathscr{C}_{static}(n) = O(n^d)$.

4.5. Regularized Topological Evolution

Neuronal adaptation with continuous gating [2]:

$$g_i(t) = \text{sigmoid}\left(\gamma \int_0^t \left(\mathbb{E}[\|\nabla_x \phi^{(i)}\|] - \text{Var}[\phi^{(i)}]\right) d\tau\right) \tag{8}$$

Output becomes $\widetilde{\phi^{(i)}} = g_i \cdot \phi^{(i)}$ with $g_i \in [0,1]$.

5. Stability and Convergence Guarantees

5.1. Lyapunov Stability for Inference

Assumption 1. *Loss* $\mathscr{L}$ *satisfies the Polyak-Łojasiewicz condition in* θ_s *during inference [11,20].*

Theorem 2 (Inference Stability)**.** *The inference dynamics* $d\theta_s/dt = -\alpha(\partial\mathscr{L}/\partial\theta_s + \partial\mathscr{S}/\partial\theta_s)$ *admit Lyapunov function [11]:*

$$V(\theta_s) = \mathscr{L}(\theta_s) + \frac{1}{2}\|\nabla_{\theta_s}\mathscr{S}\|^2 \tag{9}$$

satisfying $dV/dt \leq -\gamma V$ *for* $\gamma = \min(\alpha\lambda, \beta^{-1})$.

Proof. Through Lyapunov drift analysis [2]:

$$\begin{aligned}\frac{dV}{dt} &= \nabla_{\theta_s}\mathscr{L}^T\dot{\theta}_s + \nabla_{\theta_s}\mathscr{S}^T\nabla^2_{\theta_s}\mathscr{S}\dot{\theta}_s \\ &= -\alpha\left(\|\nabla_{\theta_s}\mathscr{L}\|^2 + \beta\|\nabla_{\theta_s}\mathscr{S}\|^2 + \nabla_{\theta_s}\mathscr{L}^T\nabla_{\theta_s}\mathscr{S} + \nabla_{\theta_s}\mathscr{S}^T\nabla^2_{\theta_s}\mathscr{S}\nabla_{\theta_s}\mathscr{L}\right) \\ &\leq -\alpha\lambda\|\nabla_{\theta_s}\mathscr{L}\|^2 - \alpha\beta^{-1}\|\nabla_{\theta_s}\mathscr{S}\|^2 \leq -\gamma V\end{aligned}$$

where the inequality follows from the PL condition and smoothness assumptions. □

5.2. Convergence Under Alternating Minimization

Theorem 3. *The bilevel optimization converges to* (θ^*, θ_s^*) *satisfying [14]:*

$$\|\nabla_\theta\mathscr{L}\| \leq \varepsilon_1, \quad \|\mathscr{D}_\varepsilon\mathscr{L}\| \leq \varepsilon_2 \tag{10}$$

at rate $O(1/\sqrt{K})$ *for K iterations, where:*

- $\mathscr{D}_\varepsilon\mathscr{L}$*: Directional derivative of loss in structural update direction*
- $\varepsilon_1, \varepsilon_2 > 0$*: Error tolerances defining near-stationary point*

6. Model Structure

To ground the theoretical framework that follows, we first define the neural model upon which the Adaptive State-Space Transfer Function (ASSTF) is built. The ASSTF is designed as a drop-in replacement for a standard neuron within a feedforward network, making the framework broadly applicable.

6.1. Network Architecture

We consider a standard L-layer fully-connected neural network. The key modification is that each neuron in the hidden layers is replaced by an ASSTF node. The forward pass for the l-th layer is defined as:

$$\mathbf{h}^{(l)} = \sigma\left(\phi^{(l)}(\mathbf{h}^{(l-1)})\right), \tag{11}$$

where $\mathbf{h}^{(l-1)}$ is the input from the previous layer, $\phi^{(l)}$ represents the vector of ASSTF outputs for all neurons in layer l (as defined in Eq. (2)), and σ is a fixed, non-parametric activation function (e.g., ReLU). This structure demonstrates that our framework generalizes standard neural networks; setting $\Psi(\cdot) = 1$ and $g_i(t) = 1$ recovers a traditional neuron.

6.2. The ASSTF Neuron and Gating Justification

The core component is the ASSTF neuron (Definition 1). Its design, featuring a base function Γ modulated by a structural modulator Ψ, is intended to decouple the neuron's core transformation from its topological state-space. The gating function $g_i(t)$ (Eq. (8)) is an *optional* mechanism introduced for networks where dynamic, input-driven feature selection is beneficial [9]. It acts as a form of *local regularization*, allowing the network to smoothly prune neurons that become redundant or noisy over time, preventing overfitting and enhancing representational efficiency. Its inclusion is a practical choice for certain applications and not a fundamental requirement of the ASSTF theory itself. The subsequent theoretical analysis holds for the core $\Gamma \circledast \Psi$ transformation regardless of the gating mechanism.

7. Training Framework and Inference Evolution

7.1. Bilevel Optimization with Continuous Relaxation and Pseudo Code

The bilevel optimization process outlined here requires the gradient of the loss with respect to the structural parameters θ_s, i.e., $\nabla_{\theta_s}\mathscr{L}$. A key challenge is that structural changes often involve non-differentiable operations. To ensure end-to-end differentiability, we employ a continuous relaxation strategy.

The structural parameters θ_s are defined as continuous proxies for architectural decisions. For instance, the rank-controlling threshold α in Eq. (4) and the gating variables $g_i(t)$ in Eq. (8) are inherently continuous and differentiable. Consequently, the operator $\mathscr{D}_\varepsilon$ in Phase 2 is simply the standard gradient:

$$\mathscr{D}_\varepsilon\mathscr{L}(\theta_s) \equiv \nabla_{\theta_s}\mathscr{L} \tag{12}$$

For any remaining non-differentiable operations (e.g., sampling from a categorical distribution of activation functions), we employ the straight-through estimator (STE) [23] to compute approximate gradients, ensuring the structural parameters can be updated via gradient descent.

$$\theta_s^{(k+1)} = \theta_s^{(k)} - \eta_2\nabla_{\theta_s}\mathscr{L} \tag{13}$$

Algorithm 1 ASSTF Bilevel Optimization

Initialize θ, θ_s
for $k = 1$ to K **do**
    Phase 1 (Core parameters):
    $\theta^{(k+1)} = \theta^{(k)} - \eta_1\nabla_\theta\mathscr{L}$
    Phase 2 (Structural parameters):
    $\theta_s^{(k+1)} = \theta_s^{(k)} - \eta_2\mathscr{D}_\varepsilon\mathscr{L}(\theta_s)$
end for

7.2. Inference-Phase Evolution

During inference, structural updates follow surprise minimization:

$$\frac{d\theta_s}{dt} = -\alpha \left(\frac{\partial \mathcal{L}}{\partial \theta_s} + \beta \frac{\partial \mathscr{S}}{\partial \theta_s} \right) \tag{14}$$

where $\mathscr{S}$ is the surprise objective:

$$\mathscr{S} = -\log p(\mathbf{x}|\theta_s) + \mathrm{KL}(q(\theta_s) \| p(\theta_s)) \tag{15}$$

This natural unsupervised objective enables the network to act as a generative model of its input stream, preventing hallucination through KL regularization.

7.3. Convergence Guarantees

Theorem 4. *Under Lipschitz continuity of* Ψ *and bounded gradients, the alternating optimization scheme converges to* (θ^*, θ_s^*) *satisfying:*

$$\|\nabla_\theta \mathscr{L}(\theta^*, \theta_s^*)\| \leq \varepsilon_1, \quad \|\mathscr{D}_\varepsilon \mathscr{L}(\theta^*, \theta_s^*)\| \leq \varepsilon_2 \tag{16}$$

Proof of Theorem 4. We analyze the convergence of the alternating minimization scheme (Algorithm 1). The proof proceeds in two steps, corresponding to the two phases of the update.

Let $\mathscr{L}^k = \mathscr{L}(\theta^{(k)}, \theta_s^{(k)})$ denote the loss at iteration k.

Step 1: Core Parameter Convergence (Phase 1). The update $\theta^{(k+1)} = \theta^{(k)} - \eta_1 \nabla_\theta \mathscr{L}$ is standard gradient descent. Under the assumption of bounded gradients ($\|\nabla_\theta \mathscr{L}\| \leq G_1$), which follows from the Lipschitz continuity of Ψ and the loss, we have for a sufficiently small learning rate η_1:

$$\mathscr{L}^{k+1} \leq \mathscr{L}^k - \frac{\eta_1}{2} \|\nabla_\theta \mathscr{L}^k\|^2.$$

Summing this inequality over k and rearranging leads to the first result:

$$\min_{0 \leq k \leq K} \|\nabla_\theta \mathscr{L}^k\|^2 \leq \frac{2(\mathscr{L}^0 - \mathscr{L}^*)}{\eta_1 K} \implies \|\nabla_\theta \mathscr{L}^k\| \leq O(1/\sqrt{K}).$$

Thus, for any $\varepsilon_1 > 0$, there exists $K_1 = O(1/\varepsilon_1^2)$ such that $\|\nabla_\theta \mathscr{L}^{K_1}\| \leq \varepsilon_1$.

Step 2: Structural Parameter Convergence (Phase 2). The structural update uses a directional derivative $\mathscr{D}_\varepsilon \mathscr{L}$, which is essentially a projected gradient. The update is $\theta_s^{(k+1)} = \theta_s^{(k)} - \eta_2 \mathscr{D}_\varepsilon \mathscr{L}(\theta_s^{(k)})$. Given the Lipschitz continuity of Ψ and the bounded gradients assumption ($\|\mathscr{D}_\varepsilon \mathscr{L}\| \leq G_2$), this update can be viewed as a descent method on a regularized version of the loss. A similar analysis to Step 1 shows that:

$$\min_{0 \leq k \leq K} \|\mathscr{D}_\varepsilon \mathscr{L}(\theta_s^{(k)})\|^2 \leq \frac{C}{K} \implies \|\mathscr{D}_\varepsilon \mathscr{L}(\theta_s^{(k)})\| \leq O(1/\sqrt{K}).$$

for some constant C. Therefore, for any $\varepsilon_2 > 0$, there exists $K_2 = O(1/\varepsilon_2^2)$ such that $\|\mathscr{D}_\varepsilon \mathscr{L}(\theta_s^{(K_2)})\| \leq \varepsilon_2$.

Combining both steps and letting $K = \max(K_1, K_2) = O(1/\min(\varepsilon_1^2, \varepsilon_2^2))$, we conclude that after K iterations the algorithm reaches a point $(\theta^{(K)}, \theta_s^{(K)})$ that satisfies the ε-approximate stationarity conditions:

$$\|\nabla_\theta \mathscr{L}(\theta^{(K)}, \theta_s^{(K)})\| \leq \varepsilon_1 \quad \text{and} \quad \|\mathscr{D}_\varepsilon \mathscr{L}(\theta_s^{(K)})\| \leq \varepsilon_2.$$

This completes the proof. □

8. Conclusion

This ASSTF framework provides: (1) Continuous state-space transitions via soft rank adaptation [13], (2) Formal capacity metrics [22], and (3) Provable inference stability [11]. This addresses fundamental limitations in adaptive neural architectures.

References

[1] Rumelhart, D.E., Hinton, G.E., & Williams, R.J. (1986). *Learning representations by back-propagating errors*. Nature, 323(6088), 533-536.

[2] Cohen, J., Hasani, R., & Lechner, M. (2020). *Stability Analysis of Adaptive Neural Controllers*. Advances in Neural Information Processing Systems, 33, 432-443.

[3] Zoph, B., & Le, Q. V. (2016). *Neural architecture search with reinforcement learning*. arXiv preprint arXiv:1611.01578.

[4] Elsken, T., Metzen, J. H., & Hutter, F. (2019). *Neural architecture search: A survey*. The Journal of Machine Learning Research, 20(1), 1997-2017.

[5] Liu, H., Simonyan, K., & Yang, Y. (2018). *Darts: Differentiable architecture search.* arXiv preprint arXiv:1806.09055.

[6] Apicella, A., Donnarumma, F., Isgrò, F., & Prevete, R. (2021). *A survey on modern trainable activation functions*. Neural Networks, 138, 14-32.

[7] Ha, D., Dai, A., & Le, Q. V. (2016). *Hypernetworks*. arXiv preprint arXiv:1609.09106.

[8] Teerapittayanon, S., McDanel, B., & Kung, H. T. (2016). *Branchynet: Fast inference via early exiting from deep neural networks*. In 2016 23rd International Conference on Pattern Recognition (ICPR) (pp. 2464-2469). IEEE.

[9] Veit, A., & Belongie, S. (2018). *Convolutional networks with adaptive inference graphs*. In Proceedings of the European Conference on Computer Vision (ECCV) (pp. 3-18).

[10] Wang, X., Yu, F., Dou, Z. Y., Darrell, T., & Gonzalez, J. E. (2018). *Skipnet: Learning dynamic routing in convolutional networks*. In Proceedings of the European Conference on Computer Vision (ECCV) (pp. 409-424).

[11] Lyapunov, A.M. (1992). *The general problem of the stability of motion*. International Journal of Control, 55(3), 531-534. (Original work published 1892)

[12] Fazel, M., Candès, E., Recht, B., & Parrilo, P. (2013). *Rank minimization and applications in system identification*. arXiv preprint arXiv:1306.6359.

[13] Fazel, M., Pong, T.K., Sun, D., & Tseng, P. (2013). *Rank minimization and applications in system identification*. arXiv preprint arXiv:1306.6359.

[14] Hong, M., Wai, H.T., Wang, Z., & Yang, Z. (2023). *Bilevel Optimization: Convergence Analysis and Enhanced Design*. SIAM Review, 65(2), 366-404.

[15] Chen, R. T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). *Neural ordinary differential equations*. Advances in neural information processing systems, 31.

[16] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., ... & Hadsell, R. (2017). *Overcoming catastrophic forgetting in neural networks*. Proceedings of the National Academy of Sciences, 114(13), 3521-3526.

[17] Halmos, P.R. (1974). *Measure Theory*. Springer-Verlag.

[18] Candès, E.J., & Recht, B. (2009). *Exact matrix completion via convex optimization*. Foundations of Computational mathematics, 9(6), 717-772.

[19] Recht, B., Fazel, M., & Parrilo, P.A. (2010). *Guaranteed minimum-rank solutions of linear matrix equations via nuclear norm minimization*. SIAM review, 52(3), 471-501.

[20] Polyak, B.T. (1963). *Gradient methods for minimizing functionals*. Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki, 3(4), 643-653.

[21] Cai, J.-F., Candès, E. J., & Shen, Z. (2010). *A singular value thresholding algorithm for matrix completion*. SIAM Journal on Optimization, 20(4), 1956-1982.

[22] Halmos, P. R. (1974). *Measure Theory*. Springer.

[23] Bengio, Y., Léonard, N., & Courville, A. (2013). *Estimating or propagating gradients through stochastic neurons for conditional computation*. arXiv preprint arXiv:1308.3432.